Stler Sudoku

Technical Specification

Version 1.3 (versionCode 5)

Platform Android 8.0+ (API 26 – 36)

App ID io.github.juliasivridi.sudoku

Repository github.com/JuliaSivridi/Sudoku

Stack Kotlin · Jetpack Compose · Material 3 · MVVM · SharedPreferences

Architecture note
Fully offline, local-only app. No network requests, no accounts, no backend. All state lives in ViewModel StateFlow; all persistence is SharedPreferences JSON. A single puzzle is always uniquely solvable — guaranteed by a `countSolutions(limit=2)` check during generation.

Table of Contents

1. Overview
2. Tech Stack
3. Architecture
4. Package / Folder Structure
5. Data Model
6. Database / Storage Schema
7. First-Launch Setup
8. Screens / Pages
9. Key Components
10. Theme & Colors
11. Navigation
12. CI/CD & Build
13. First-Time Developer Setup
14. Key Algorithms

1. Overview

Stler Sudoku is a local-only Android puzzle game. The player solves 9×9 Sudoku grids at three difficulty levels, aided by optional pencil marks (Notes), an auto-fill candidates toggle (Clues), and a hint system. There is no network layer, no accounts, and no backend.

Decision	Choice	Rationale
UI toolkit	Jetpack Compose (Material 3)	Modern declarative UI; eliminates XML layouts
State management	ViewModel + StateFlow	Lifecycle-aware, unidirectional data flow, testable
Theme propagation	`CompositionLocal` (`LocalAppThemeColors`)	Per-theme color access without threading colors through every composable
Persistence	SharedPreferences + `org.json`	No relational data; simple key-value and one serialised object suffice
Input mode	Two modes switchable in Settings	Supports digit-first and cell-first play styles without cluttering the game UI
Puzzle uniqueness	Backtracking solver with `countSolutions(limit=2)`	Guarantees exactly one solution per generated puzzle
No timer / errors	Intentionally omitted	Design choice: relaxed, pressure-free gameplay
No network	Fully offline	No server cost, no auth complexity, works without connectivity

2. Tech Stack

Layer	Library / Tool	Version	Notes
Language	Kotlin	2.2.10 (plugin)	JVM target 11
UI	Jetpack Compose BOM	2024.09.00	Pins all Compose library versions
UI	Compose Material 3	(from BOM)	Components, color scheme, typography
UI	Material Icons Extended	(from BOM)	AutoAwesome, Lightbulb, EditNote, Undo, etc.
Navigation	Navigation Compose	2.7.7	Typed routes via `Routes` object
Lifecycle	Lifecycle Runtime KTX	2.6.1	`viewModelScope`, `collectAsState`
Lifecycle	Lifecycle ViewModel Compose	2.7.0	`viewModel()` factory in composables
Activity	Activity Compose	1.8.0	`setContent {}` entry point
Core	Core KTX	1.10.1	Extensions for `Context`, `SharedPreferences`
Build	Android Gradle Plugin	9.1.0	`com.android.application`
Build	Kotlin Compose Plugin	2.2.10	Enables Compose compiler
Persistence	SharedPreferences + `org.json`	Platform	No third-party database
Min SDK	Android 8.0 (API 26)	—	Adaptive icons support
Target SDK	API 36	—	
Test	JUnit 4	4.13.2	Unit tests (boilerplate only)
Test	Espresso Core	3.5.1	Instrumented tests (boilerplate only)

3. Architecture

Pattern: MVVM with unidirectional data flow.

```
User gesture
     │
     ▼
Composable (UI layer)
     │  calls method on
     ▼
ViewModel  ──reads/writes──►  Data Manager (SharedPreferences)
     │
     │  emits new state via StateFlow
     ▼
collectAsState()
     │
     ▼
Composable re-renders
```

Write path — example: user places a digit

1. User taps a cell → `GameScreen` receives via `onCellTap` or `NumberRow.onDigitSelected`
2. Based on `InputPreference`, routes to `viewModel.onCellTap()` or `viewModel.placeDigit()`
3. ViewModel mutates a copy of `GameState`, pushes to `_state` (StateFlow)
4. The `init {}` collector fires auto-save to `GameSaveManager`
5. `collectAsState()` in `GameScreen` delivers the new state to `SudokuGrid`, `ControlButtons`, `NumberRow`

Read path — example: Continue button on Start screen

1. `StartScreen` calls `GameSaveManager.hasSavedGame()` on composition
2. If `true`, the Continue button is rendered
3. On tap, navigates to the `GAME` route with `loadSaved=true`
4. `GameViewModel.loadSavedGame()` reads JSON and restores `GameState`

Error handling

- `GameSaveManager`: serialisation errors are silently swallowed; `loadGame()` returns `null` → fresh game starts
- `SudokuGenerator`: `removeCells` only removes a cell after uniqueness is confirmed; every puzzle is always valid
- No network layer → no retry logic needed

ViewModels and their consumers

ViewModel	Extends	State	Consumers
`GameViewModel`	`AndroidViewModel`	`StateFlow<GameState>`	`GameScreen`, `SudokuGrid`, `ControlButtons`, `NumberRow`
`SolverViewModel`	`ViewModel`	`StateFlow<SolverState>`	`SolverScreen`
`InputPreferenceViewModel`	`AndroidViewModel`	`StateFlow<InputPreference>`	`AppNavigation` → `GameScreen`, `SettingsScreen`
`ThemeViewModel`	`AndroidViewModel`	`StateFlow<AppColorTheme>`	`MainActivity`, `AppNavigation`, `SettingsScreen`

> `GameViewModel` and `SolverViewModel` are scoped to the `NavHost` (created once in `AppNavigation`) and survive intra-app navigation. `ThemeViewModel` and `InputPreferenceViewModel` are created in `MainActivity` and passed down as parameters.

4. Package / Folder Structure

```
app/src/main/
├── AndroidManifest.xml              ← single Activity, portrait-only, edge-to-edge
└── java/com/example/sudoku/
    ├── MainActivity.kt              ← creates ThemeVM + InputPrefVM; hosts SudokuTheme +
AppNavigation
    |
    ├── model/
    |   ├── Cell.kt                 ← Cell data class (value, isGiven, notes)
    |   ├── Difficulty.kt            ← enum EASY/MEDIUM/HARD with givens count
    |   ├── GameState.kt             ← GameState + InputMode enum
    |   └── InputPreference.kt       ← enum NUMBER_FIRST / CELL_FIRST
    |
    ├── game/
    |   ├── SudokuGenerator.kt       ← random board fill + uniqueness-checked cell
removal
    |   └── SudokuSolver.kt          ← solve(), countSolutions(), isValid()
    |
    ├── data/
    |   ├── GameSaveManager.kt       ← JSON serialise/deserialise GameState ↔
SharedPreferences
    |   ├── StatsManager.kt          ← per-difficulty completed-game counters
    |   ├── InputPreferenceManager.kt ← persists InputPreference enum name
    |   └── ThemeManager.kt          ← persists AppColorTheme enum name
    |
    ├── viewmodel/
    |   ├── GameViewModel.kt         ← all game logic (AndroidViewModel)
    |   ├── SolverViewModel.kt       ← standalone solver screen logic
    |   ├── InputPreferenceViewModel.kt ← thin wrapper over InputPreferenceManager
    |   └── ThemeViewModel.kt        ← thin wrapper over ThemeManager
    |
    ├── navigation/
    |   └── AppNavigation.kt        ← NavHost; Routes object; shared VM creation
    |
    └── ui/
        ├── theme/
        |   ├── Color.kt             ← all color constants; AppThemeColors; 4 palettes
        |   ├── Theme.kt             ← SudokuTheme; LocalAppThemeColors CompositionLocal
        |   └── Type.kt              ← Typography (bodyLarge only customised)
        ├── components/
        |   ├── SudokuGrid.kt          ← 9×9 grid with canvas borders, highlights, notes
        |   ├── ControlButtons.kt      ← Undo / Clear / Notes / Clues / Hint row
        |   └── NumberRow.kt           ← 3×3 digit picker with count-based disable
        └── screens/
            ├── StartScreen.kt        ← home screen + BottomNavBar + MenuButton
```

```
        ├── GameScreen.kt        ← active game; routes input by InputPreference
        ├── EndScreen.kt         ← completion screen with motivational message
        ├── SolverScreen.kt      ← manual solver; reuses SudokuGrid
        ├── StatisticsScreen.kt  ← per-difficulty completed counts
        └── SettingsScreen.kt    ← input mode toggle + theme selector
```

```
/ (project root)
├── build.gradle.kts              ← top-level; plugins apply false
├── settings.gradle.kts           ← module inclusion; repository declarations
├── gradle/libs.versions.toml     ← version catalog for all dependencies
├── gradle.properties             ← -Xmx2048m; kotlin.code.style=official
├── app/build.gradle.kts          ← SDK config; signing config; dependencies
└── .github/workflows/release.yml ← CI/CD: APK + AAB release pipeline
```

5. Data Model

Cell

Field	Type	Default	Description
`value`	`Int`	`0`	`0` = empty; `1–9` = digit
`isGiven`	`Boolean`	`false`	Pre-filled clue; user cannot modify
`notes`	`Set<Int>`	`emptySet()`	Pencil-mark candidates `1–9`

Difficulty

Name	label	givens	Cells removed
`EASY`	"Easy"	46	35
`MEDIUM`	"Medium"	36	45
`HARD`	"Hard"	29	52

InputMode (enum inside GameState.kt)

Value	Description
`NORMAL`	Tapping a cell places the selected digit
`ERASE`	Tapping a cell clears it (Number-First mode only)
`NOTES`	Tapping a cell toggles the selected digit as a pencil mark

GameState

Field	Type	Default	Description
`board`	`List<List<Cell>>`	9×9 of Cell()	Live board
`solution`	`List<List<Int>>`	9×9 of 0	Correct answer; used for hints and completion check
`difficulty`	`Difficulty`	EASY	Active difficulty level

Field	Type	Default	Description
`selectedCell`	`Pair<Int,Int>?`	null	(row, col) of highlighted cell
`selectedDigit`	`Int?`	null	Active digit 1–9
`inputMode`	`InputMode`	NORMAL	Active input mode
`isComplete`	`Boolean`	false	true when all 81 cells match solution
`autoNotesActive`	`Boolean`	false	true while Clues mode is on
`undoStack`	`List<List<List<Cell>>>`	emptyList()	Board snapshots; max 50 entries

> **Note:** `undoStack` stores board layouts only — not `inputMode`, `selectedCell`, or `autoNotesActive`. Undoing restores the board but leaves the Clues toggle state unchanged (intentional decoupling).

InputPreference

Value	Description
`NUMBER_FIRST`	Select digit in NumberRow → tap cell to place
`CELL_FIRST`	Tap cell to select → tap digit in NumberRow to place

SolverState

Field	Type	Default	Description
`board`	`List<List<Cell>>`	9×9 of Cell()	Manually entered puzzle
`selectedDigit`	`Int?`	null	Active digit in digit picker
`isClearMode`	`Boolean`	false	When true, tapping a cell clears it
`isSolved`	`Boolean`	false	true after successful solve
`noSolution`	`Boolean`	false	true when solver finds no solution
`undoStack`	`List<List<List<Cell>>>`	emptyList()	Board snapshots

AppColorTheme & AppThemeColors

AppColorTheme value	displayName
`ORANGE`	"Orange"
`GREEN`	"Green"
`BLUE`	"Blue"
`PURPLE`	"Purple"

AppThemeColors field	Type	Description
`accent`	`Color`	Primary accent — buttons, selected digit text, titles
`accentVariant`	`Color`	Lighter accent — user digit color in dark theme
`cellDigitHighlightLight`	`Color`	Same-digit cell tint, light theme
`cellDigitHighlightDark`	`Color`	Same-digit cell tint, dark theme
`userNumberLight`	`Color`	User-placed digit text color, light theme
`userNumberDark`	`Color`	User-placed digit text color, dark theme

6. Database / Storage Schema

All persistence uses Android `SharedPreferences`. There is no SQLite database, no files, and no network storage.

File: `"sudoku_prefs"`

Key	Type	Default	Written by	Purpose
`"saved_game"`	String (JSON)	absent	`GameSaveManager`	Serialised in-progress `GameState`; removed on completion
`"input_preference"`	String	`"NUMBER_FIRST"`	`InputPreferenceManager`	Persisted `InputPreference.name`
`"color_theme"`	String	`"ORANGE"`	`ThemeManager`	Persisted `AppColorTheme.name`

File: `"sudoku_stats"`

Key	Type	Default	Written by	Purpose
`"EASY"`	Int	0	`StatsManager`	Completed Easy game count
`"MEDIUM"`	Int	0	`StatsManager`	Completed Medium game count
`"HARD"`	Int	0	`StatsManager`	Completed Hard game count

JSON schema — `"saved_game"` value

```
{
  "difficulty": "EASY",          // "EASY" | "MEDIUM" | "HARD"
  "board": [                     // 9 rows
    [                            // 9 columns
      { "v": 5, "g": true,  "n": [] },          // given clue
      { "v": 0, "g": false, "n": [1, 3, 7] },   // empty cell with notes
      ...
    ],
    ...
  ],
  "solution": [                  // 9×9 array of plain Int
    [5, 3, 4, 6, 7, 8, 9, 1, 2],
    ...
```

```
  ]
}
```

JSON key	Source field	Type
`"v"`	`Cell.value`	Int 0–9
`"g"`	`Cell.isGiven`	Boolean
`"n"`	`Cell.notes`	JSON array of Int
`"difficulty"`	`Difficulty.name`	"EASY" / "MEDIUM" / "HARD"

> **Not serialised:** `selectedCell`, `selectedDigit`, `inputMode`, `isComplete`, `undoStack`, `autoNotesActive`. All reset to defaults on load.
>
> **Auto-save trigger** (in `GameViewModel.init {}`): save fires whenever any non-given cell has `value != 0`; save is *removed* when `isComplete == true`.

7. First-Launch Setup

There is no user authentication, no onboarding wizard, and no remote configuration. The app is fully functional immediately on first launch.

Condition	Behaviour
`ThemeManager` finds no `"color_theme"`	Defaults to `ORANGE`
`InputPreferenceManager` finds no key	Defaults to `NUMBER_FIRST`
`GameSaveManager.hasSavedGame()` returns false	Continue button not shown on Start screen
`StatsManager` finds no keys	Returns 0 for all difficulties
User selects a difficulty	Game starts immediately

No permissions are requested. No Google Play Services or Firebase dependencies. No network requests ever made.

8. Screens / Pages

StartScreen

File: `ui/screens/StartScreen.kt` · **Route:** `"start"` (start destination)
ViewModels: none — reads `GameSaveManager` directly via `LocalContext`

Element	Details
Title	"SUDOKU" · 28sp · accent color
Continue button	Shown only if `hasSavedGame() == true`; placed above difficulty group; 56dp height
Difficulty buttons	Easy / Medium / Hard — `MenuButton` composable; 56dp height; `RoundedCornerShape(12dp)`
Solver button	Outlined `Button`; navigates to `SOLVER` route
Spacers	~56dp between Continue and difficulty group; ~56dp between difficulty group and Solver
Bottom bar	`BottomNavBar` with home tab selected

GameScreen

File: `ui/screens/GameScreen.kt` · **Route:** `"game/{difficulty}?loadSaved={loadSaved}"`
ViewModels: `GameViewModel` · **Nav args:** `difficulty: String`, `loadSaved: Boolean` (default false)

On `LaunchedEffect(unit)`: if `loadSaved == true` → `viewModel.loadSavedGame()`; else → `viewModel.startGame(difficulty)`.
On `LaunchedEffect(state.isComplete)`: if `isComplete == true` → calls `onGameComplete()`.

Event	NUMBER_FIRST	CELL_FIRST
Cell tapped	`viewModel.onCellTap(row, col)`	`viewModel.selectCell(row, col)`
Digit tapped	`viewModel.selectDigit(digit)`	`viewModel.placeDigit(digit)`
Clear button	`viewModel.toggleErase()`	`viewModel.eraseSelected()`
`NumberRow.showSelection`	`true`	`false`

EndScreen

File: `ui/screens/EndScreen.kt` · **Route:** `"end/{difficulty}"` · **ViewModels:** none

Displays a 🎉 emoji (72sp), "Puzzle Complete!" (28sp bold), one of 6 random motivational messages, "Play Again" and "Menu" buttons.

SolverScreen

File: `ui/screens/SolverScreen.kt` · **Route:** `"solver"` · **ViewModels:** `SolverViewModel`

User manually enters digits of an unknown puzzle. `SudokuGrid` renders with `isSolverMode=true` — conflict highlighting suppressed; all digits shown in accent color. Tapping "Solve" runs the backtracking solver. "No solution found." is shown in red when `state.noSolution == true`.

StatisticsScreen

File: `ui/screens/StatisticsScreen.kt` · **Route:** `"statistics"` · **ViewModels:** none

Shows completed-game counts in a bordered table (Mode | Completed). Counts are read once at composition — NOT reactive. `BottomNavBar` with stats tab selected.

SettingsScreen

File: `ui/screens/SettingsScreen.kt` · **Route:** `"settings"`
State sources: `InputPreferenceViewModel`, `ThemeViewModel` (passed as params from `AppNavigation`)

Section	Component	Behaviour
Input mode	`SingleChoiceSegmentedButtonRow`	"Number first" / "Cell first"; persisted immediately via `InputPreferenceViewModel`
Theme	One full-width button per `AppColorTheme`	Button background = palette's `accent`; checkmark icon on active theme; persisted via `ThemeViewModel`
Bottom bar	`BottomNavBar`	Settings tab selected

9. Key Components

SudokuGrid ui/components/SudokuGrid.kt

Parameter	Type	Default	Purpose
`state`	`GameState`	—	Full game state
`onCellTap`	`(Int,Int)->Unit`	—	Callback (row, col) on cell tap
`isSolverMode`	`Boolean`	false	Suppresses conflict highlighting; all cells treated as user-placed

Cell background priority (highest wins)

Priority	Condition	Background
1	Cell is selected	`CellSelected*`
2	Cell's value matches `selectedDigit`	`appColors.cellDigitHighlight*`
3	Same row or column as selected cell	`CellHighlight*`
4	Default	Transparent

Conflict overlay (separate pass): `ConflictBg*` background + `ConflictColor*` text, applied for cells where `hasConflict == true`.

Border rendering via `Modifier.drawWithContent`: inner cell lines 1dp; block boundary lines 2dp; outer frame 2dp on all 4 sides.

Notes sub-grid: private `NotesGrid` composable; 3×3 layout; 8sp font; `#909090` light / `#9E9E9E` dark. Rendered only when `cell.value == 0 && cell.notes.isNotEmpty()`.

Text sizes: 24sp for digits; 8sp for note marks.

ControlButtons ui/components/ControlButtons.kt

Parameter	Type	Default	Purpose
`inputMode`	`InputMode`	—	Current input mode
`isCellFirst`	`Boolean`	false	Changes Clear button active-state logic
`isAutoNotesActive`	`Boolean`	false	Drives Clues button highlight

Parameter	Type	Default	Purpose
`onUndo`	`()->Unit`	—	
`onToggleErase`	`()->Unit`	—	
`onToggleNotes`	`()->Unit`	—	
`onToggleAutoNotes`	`()->Unit`	—	
`onHint`	`()->Unit`	—	

#	Label	Icon	isActive condition
1	Undo	`AutoMirrored.Filled.Undo`	always false
2	Clear	`Filled.Close`	`!isCellFirst && inputMode == ERASE`
3	Notes	`Filled.EditNote`	`inputMode == NOTES`
4	Clues	`Outlined.AutoAwesome`	`isAutoNotesActive`
5	Hint	`Outlined.Lightbulb`	always false

Active: `primary` (accent) tint · `FontWeight.SemiBold` · 12sp label. Inactive: `onBackground` · `FontWeight.Normal`. Icon size: 28dp. Vertical padding: 10dp. Layout: `SpaceEvenly` Row, each button `weight(1f)`.

NumberRow ui/components/NumberRow.kt

Parameter	Type	Purpose
`board`	`List<List<Cell>>`	Used to compute per-digit occurrence counts
`selectedDigit`	`Int?`	Currently highlighted digit
`showSelection`	`Boolean`	`false` in CELL_FIRST — digits tappable but never visually highlighted
`onDigitSelected`	`(Int)->Unit`	Fires when user taps a non-full digit

Layout: 3-column × 3-row grid; outer `border(1dp, GridInnerBorder*)`. Each digit cell: `weight(1f)` + `aspectRatio(2f)` (width = 2× height). Font: 32sp.
Disable: digit is "full" when count on board ≥ 9 — text hidden, cell not clickable.
Selection highlight (when `showSelection == true`): `cellDigitHighlight*` background · `accent` text · `FontWeight.Bold`.

BottomNavBar ui/screens/StartScreen.kt

Parameter	Type	Purpose
`currentTab`	`String`	"home" / "stats" / "settings"
`onHomeSelected`	`()->Unit`	
`onStatisticsSelected`	`()->Unit`	
`onSettingsSelected`	`()->Unit`	

Three `NavigationBarItem`s: Home (`Outlined.Home`), Stats (`Outlined.Leaderboard`), Settings (`Outlined.Settings`). Icon: 28dp. Selected: `accent` ; unselected: `onBackground @ 60% alpha` ; indicator: `Transparent` .

10. Theme & Colors

Named Color Constants (ui/theme/Color.kt)

Constant	Hex	Usage
`Orange`	#E07E38	Default accent color
`OrangeLight`	#E8935A	Default accent variant
`BackgroundLight`	#FFFFFF	App background, light theme
`SurfaceLight`	#FFFFFF	Surface, light theme
`OnBackgroundLight`	#1C1C1C	Text / icons on light background
`BackgroundDark`	#1C1C1C	App background, dark theme
`SurfaceDark`	#363636	Surface, dark theme
`OnBackgroundDark`	#EEEEEE	Text / icons on dark background
`GridOuterBorderLight`	#4D5666	3×3 block boundaries + outer grid frame, light
`GridInnerBorderLight`	#BEC5D1	Inner cell dividers, light
`GridOuterBorderDark`	#656D7A	3×3 block boundaries + outer grid frame, dark
`GridInnerBorderDark`	#363C47	Inner cell dividers, dark
`CellHighlightLight`	#F0F0F0	Row / column highlight background, light
`CellHighlightDark`	#363636	Row / column highlight background, dark
`CellSelectedLight`	#E4E4E4	Selected cell background, light
`CellSelectedDark`	#424242	Selected cell background, dark
`CellDigitHighlightLight`	#33CF7A30	Same-digit cell tint (~20% alpha), light
`CellDigitHighlightDark`	#80CF7A30	Same-digit cell tint (~50% alpha), dark
`GivenNumberLight`	#1C1C1C	Pre-filled clue digit text, light
`GivenNumberDark`	#EEEEEE	Pre-filled clue digit text, dark
`UserNumberLight`	#E07E38	User-placed digit text, light
`UserNumberDark`	#E8935A	User-placed digit text, dark

Constant	Hex	Usage
`NoteNumberLight`	#909090	Pencil-mark note text, light
`NoteNumberDark`	#9E9E9E	Pencil-mark note text, dark
`ConflictColorLight`	#CC1515	Conflicting digit text, light
`ConflictColorDark`	#FF5252	Conflicting digit text, dark
`ConflictBgLight`	#33CC1515	Conflict cell background (~20% alpha), light
`ConflictBgDark`	#80CC1515	Conflict cell background (~50% alpha), dark

Four Theme Palettes

Palette	accent	accentVariant	userNumberLight	userNumberDark
Orange	#E07E38	#E8935A	#E07E38	#E8935A
Green	#4D8B53	#79B57F	#4D8B53	#79B57F
Blue	#3D73B0	#6B9FD4	#3D73B0	#6B9FD4
Purple	#7A4FA3	#A47BC8	#7A4FA3	#A47BC8

> `cellDigitHighlightLight` = accent at `0x33` alpha · `cellDigitHighlightDark` = accent at `0x80` alpha

Material 3 Color Scheme Mapping

M3 token	Dark theme	Light theme
`primary`	`themeColors.accent`	`themeColors.accent`
`onPrimary`	Black	White
`secondary`	`themeColors.accentVariant`	`themeColors.accentVariant`
`background`	BackgroundDark `#1C1C1C`	BackgroundLight `#FFFFFF`
`surface`	SurfaceDark `#363636`	SurfaceLight `#FFFFFF`
`onBackground`	OnBackgroundDark `#EEEEEE`	OnBackgroundLight `#1C1C1C`

`SudokuTheme` wraps content in `CompositionLocalProvider(LocalAppThemeColors provides themeColors)`, making the full palette available to any composable via `LocalAppThemeColors.current`.

11. Navigation

Routes

Constant	URL pattern	Nav args
`START`	`"start"`	—
`GAME`	`"game/{difficulty}?loadSaved={loadSaved}"`	`difficulty: String`, `loadSaved: Boolean` (default false)
`END`	`"end/{difficulty}"`	`difficulty: String`
`SOLVER`	`"solver"`	—
`STATISTICS`	`"statistics"`	—
`SETTINGS`	`"settings"`	—

Builder helpers in `Routes`: `Routes.game(difficulty)` → `"game/${difficulty.name}?loadSaved=false"` · `Routes.continueGame()` → `"game/SAVED?loadSaved=true"` · `Routes.end(difficulty)` → `"end/${difficulty.name}"`

Back-Stack Management

Navigation event	popUpTo behaviour
Game complete → End screen	`popUpTo(START)` (exclusive)
End "Menu" → Start	`popUpTo(START) { inclusive = true }`
End "Play Again" → Game	`popUpTo(START)`, then navigate to GAME
Solver / Stats / Settings → Start	`popUpTo(START) { inclusive = true }`

There are no deeplinks. The app has no `intent-filter` for custom URI schemes.

12. CI/CD & Build

Workflow file: `.github/workflows/release.yml`

Setting	Value
Trigger	`push` on tags matching `v*`
Runner	`ubuntu-latest`
JDK	Temurin 17
Permissions	`contents: write`

Pipeline steps

#	Step	Details
1	Checkout	`actions/checkout@v4.2.2`
2	Set up JDK 17	`actions/setup-java@v4.7.0`, distribution `temurin`
3	Make gradlew executable	`chmod +x ./gradlew`
4	Decode keystore	`secrets.KEYSTORE_BASE64` → `keystore.jks` via `base64 --decode`
5	Build release APK + AAB	`./gradlew assembleRelease bundleRelease` with `KEYSTORE_PATH`, `KEYSTORE_PASSWORD`, `KEY_ALIAS`, `KEY_PASSWORD` env vars
6	Rename artifacts	`app-release.apk` → `stler-sudoku.apk`; `app-release.aab` → `stler-sudoku.aab`
7	Create GitHub Release	`softprops/action-gh-release@v2`; attaches both artifacts

> **Required secrets:** `KEYSTORE_BASE64` `KEYSTORE_PASSWORD` `KEY_ALIAS` `KEY_PASSWORD`

App build config (app/build.gradle.kts)

Setting	Value
`applicationId`	`io.github.juliasivridi.sudoku`
`compileSdk`	36

Setting	Value
`minSdk`	26
`targetSdk`	36
`versionCode`	5
`versionName`	"1.3"
`isMinifyEnabled`	false
Java source/target	VERSION_11

13. First-Time Developer Setup

1. Install Android Studio Hedgehog (2023.1) or later
2. Clone the repository:

   ```
   git clone https://github.com/JuliaSivridi/Sudoku.git
   ```

3. Open the project in Android Studio; let Gradle sync finish
4. Connect an Android device (API 26+) or create an AVD with API 26+
5. Press **Run ▶** — a debug APK builds and deploys to the device

Build a release APK locally

```
./gradlew assembleRelease \
  -PKEYSTORE_PATH=/path/to/keystore.jks \
  -PKEYSTORE_PASSWORD=*** \
  -PKEY_ALIAS=*** \
  -PKEY_PASSWORD=***
# Output: app/build/outputs/apk/release/app-release.apk
```

Publish a new release

```
# 1. Bump versionCode and versionName in app/build.gradle.kts
# 2. Commit and tag
git tag v<versionName>
git push origin v<versionName>
# GitHub Actions builds APK + AAB and creates a GitHub Release automatically
```

14. Key Algorithms

Puzzle Generation

```
generate(difficulty):
    solution ← fillBoard(emptyBoard)     // random backtracking
    puzzle   ← deepCopy(solution)
    toRemove ← 81 - difficulty.givens    // 35 / 45 / 52
    removeCells(puzzle, toRemove)
    board ← toCellGrid(puzzle)           // non-zero cells become isGiven=true
    return (board, solution)

fillBoard(board):
    (r, c) ← first cell where value == 0 (row-major scan)
    if no such cell: return true         // board fully filled
    digits ← [1..9].shuffled()          // randomise order each call
    for d in digits:
        if isValid(board, r, c, d):
            board[r][c] ← d
            if fillBoard(board): return true
            board[r][c] ← 0             // backtrack
    return false

removeCells(board, count):
    positions ← [0..80].shuffled()
    removed ← 0
    for pos in positions:
        if removed == count: break
        r ← pos / 9;  c ← pos % 9
        if board[r][c] == 0: continue   // already empty
        backup ← board[r][c]
        board[r][c] ← 0
        if countSolutions(deepCopy(board), limit=2) == 1:
            removed++                   // unique solution — keep removal
        else:
            board[r][c] ← backup        // restore — would break uniqueness
```

Puzzle Solving (Backtracking)

```
solve(board):
    (r, c) ← first cell where value == 0 (row-major)
    if no such cell: return true         // all cells filled
    for d in 1..9:
        if isValid(board, r, c, d):
            board[r][c] ← d
            if solve(board): return true
```

```
            board[r][c] ← 0
    return false

countSolutions(board, limit):
    (r, c) ← first cell where value == 0
    if no such cell: return 1            // found a complete solution
    count ← 0
    for d in 1..9:
        if isValid(board, r, c, d):
            board[r][c] ← d
            count += countSolutions(board, limit)
            board[r][c] ← 0
            if count >= limit: return count  // early exit
    return count

isValid(board, r, c, d):
    for each cell in row r    : if value == d: return false
    for each cell in column c : if value == d: return false
    br ← (r / 3) * 3;  bc ← (c / 3) * 3
    for dr in 0..2, dc in 0..2:
        if board[br+dr][bc+dc] == d: return false
    return true
```

Conflict Detection (SudokuGrid.hasConflict)

```
hasConflict(board, row, col):
    cell ← board[row][col]
    if cell.isGiven: return false        // given cells never highlighted red
    if cell.value == 0: return false     // empty cells not in conflict
    v ← cell.value
    for c in 0..8:
        if c ≠ col  and board[row][c].value == v: return true
    for r in 0..8:
        if r ≠ row  and board[r][col].value == v: return true
    br ← (row/3)*3;  bc ← (col/3)*3
    for dr in 0..2, dc in 0..2:
        r2 ← br+dr;  c2 ← bc+dc
        if (r2,c2) ≠ (row,col) and board[r2][c2].value == v: return true
    return false
```

> If a user places a digit that duplicates a given clue, only the user-placed cell turns red — the given cell is never highlighted as an error.

Auto-Notes / Clues Toggle

```
toggleAutoNotes():
    undoStack ← buildUndoStack(current)

    if not current.autoNotesActive:
```

```
        for each cell(r, c) in board:
            if cell.value == 0:
                candidates ← { d in 1..9 | canPlace(board, r, c, d) }
                newBoard[r][c] ← cell.copy(notes = candidates)
        state ← state.copy(board=newBoard, autoNotesActive=true, undoStack=undoStack)
    else:
        for each cell in board:
            newBoard[r][c] ← cell.copy(notes = emptySet())
        state ← state.copy(board=newBoard, autoNotesActive=false, undoStack=undoStack)

canPlace(board, r, c, d):
    for other cells in row r    : if .value == d: return false
    for other cells in col c    : if .value == d: return false
    for cells in 3×3 box        : if .value == d: return false
    return true
```

Note Cleanup After Digit Placement

```
cleanNotesAfterPlacement(board, row, col, digit):
    totalCount ← count of cells where .value == digit
    if totalCount >= 9:
        // digit fully placed — remove from all notes everywhere
        for every cell: notes.remove(digit)
    else:
        // remove from same row, column, and 3×3 box only
        for c in 0..8: board[row][c].notes.remove(digit)
        for r in 0..8: board[r][col].notes.remove(digit)
        br ← (row/3)*3;  bc ← (col/3)*3
        for dr in 0..2, dc in 0..2:
            board[br+dr][bc+dc].notes.remove(digit)
```

Undo Stack Management

```
buildUndoStack(state):
    return (state.undoStack + state.board).takeLast(50)

undo():
    if undoStack.isEmpty(): return
    previousBoard ← undoStack.last()
    state ← state.copy(
        board     = previousBoard,
        undoStack = undoStack.dropLast(1)
        // autoNotesActive NOT changed — Clues toggle is independent of undo
    )
```

Completion Check

```
checkComplete(state):
    for r in 0..8, c in 0..8:
        if state.board[r][c].value == 0: return false
        if state.board[r][c].value ≠ state.solution[r][c]: return false
    return true
// Called after every digit placement. On true: stops auto-save, increments stats.
```